

BLAST → DIAMOND: Konzept zur Laufzeitoptimierung

Schneller Einzelsequenz-Vergleich einer Virussequenz gegen NCBI nr

Stand: 23.09.2026 — dell-3660 (192.168.5.23)

Erzeugt von dell-3660: /home/gh/python/blast_diamond_pdf.py → /var/www/web1/blast_diamond.pdf

Inhaltsverzeichnis

1	Ausgangslage	1
2	Analyse der Ursache	1
3	Strategie: DIAMOND statt BLASTX	1
4	Stand	1
5	Append: neue Sequenzen ohne Neubau	2
6	Metadaten in MariaDB (wagodb, Präfix blast_)	3
7	Öffentliche API	5
8	AmyloDeep auf der Tesla P4	8
9	Doppelte Datenhaltung auflösen	12
10	Mögliche nächste Stufe	14

1. Ausgangslage

blast2.py führt einen blastx-Lauf einer einzelnen SARS-CoV-2-Genomsequenz (~30 kb, 6 Frames) gegen die vollständige lokale NCBI-nr-Datenbank aus, eingeschränkt per **-taxids 4751** auf Fungi. Beobachtete Laufzeit: > 20 Minuten für einen einzigen Lauf.

2. Analyse der Ursache

Kennzahl	Wert
Datenbankgröße	844 GB, 176 Volumes
Sequenzen in nr	1,15 Mrd.
Residuen gesamt	435 Mrd.
RAM (dell-3660)	62 GB
genutzte Kerne im Skript	8 von 20
GPU	Tesla P4, 8 GB VRAM, Pascal sm_61

Die Datenbank übersteigt den verfügbaren Arbeitsspeicher um mehr als das Zehnfache, der Lauf ist damit an die Platte gebunden. Der Taxid-Filter reduziert die gemeldeten Treffer, ändert aber nichts an der nötigen Wortsuche über die vollständige Datenbank. Zusätzlich wird weniger als die Hälfte der CPU-Kerne genutzt.

3. Strategie: DIAMOND statt BLASTX

DIAMOND (Buchfink, Xie & Huson, *Nature Methods* 2015; Weiterentwicklung Buchfink et al. 2021, „Sensitive protein alignments at tree-of-life scale“) ist ausdrücklich für den Vergleich einzelner Sequenzen gegen Datenbanken in nr-Größe

entwickelt. Der Kern ist eine Doppel-Indizierung (Query und Datenbank werden gemeinsam sortiert und geseedet, statt die Datenbank linear je Query zu durchsuchen) plus Spaced Seeds. Berichtete Beschleunigung gegenüber BLASTX: 100–20.000× bei vergleichbarer Trefferqualität; die Datenbank muss dabei nicht ins RAM passen (blockweises Streaming).

Dieselbe Grundidee (schneller Alignment-Vorfilter gegen komplette nr/nt zur Identifikation unbekannter/neuartiger Erreger) steckt hinter CZ-ID / IDseq (Kalantar et al. 2020, *GigaScience*), einer Pipeline des Chan Zuckerberg Biohub für genau diesen Anwendungsfall.

Alternative: MMseqs2 (Steinegger & Söding, *Nature Biotechnology* 2017) verfolgt denselben Vorfilter-Gedanken, ist bei höchster Empfindlichkeit etwas langsamer als DIAMOND, dafür teils treffsicherer bei entfernten Homologien. Eingesetzt wird DIAMOND: geringster Umstellungsaufwand (Drop-in für blast2.py), kein zusätzlicher Unterbau nötig.

4. Stand

Statt einer physischen Teil-DB (Fungi) gibt es eine vollständige `nr_full.dmnd` mit Taxonomie-Map; eingeschränkt wird bei der Suche per `--taxonlist`. Gebaut wird auf `sda5 (/mnt/archive)`.

Ein Durchgang für .dmnd und Metadaten: `~/python/nr_build.py` (seit 23.09.2026, ersetzt `diamond_pipeline.sh`, `filter_nonempty.py`, den Erstimport von `blast_meta2db.py` und `blast_release_finalize.sh`). Ein einziger `blastdbcmd`-Strom (je Define OID, Accession, TaxID, Länge, Sequenz, Titel) wird verteilt: an `makedb` mit Kopfzeilen nur aus Accessions (`>acc1 >acc2 ...`, Titel nur noch in MariaDB), an MariaDB als Chunk-Dateien je 5 Mio. Sequenzen, die ein eigener Thread lädt (Warteschlange begrenzt, bremst den Strom statt die Platte zu füllen). Leersequenzen gehen nicht in die Datenbankdatei und werden in `blast_dmnd_luecke` vermerkt. Danach Verifikation beider Ziele gegen `blastdbcmd -info`, Indizes (`tmpdir` auf der NVMe), Taxonomie, `blast_release`. Erst dann nimmt die API Suchen an.

Speicherfehler im ersten Bau (23.09.2026, 11:03): `makedb` sammelte die Offset-Tabelle (16 Byte je Sequenz) in einem `vector` (`dmnd.cpp`), der bei 2^{30} Einträgen – Volume ~164 von 176 – seine Kapazität verdoppelte und kurz ~51 GB brauchte; der OOM-Killer beendete den Bau nach 3,5 h. Patch: die Einträge gehen in 1-Mio.-Blöcken in `<db>.pos_tmp` und werden am Ende in den Trailer kopiert. Geprüft an 2,5 Mio. nr-Sequenzen: Datei byteweise identisch zum unveränderten Binary, `--append` ebenfalls identisch. Gebaut wird mit `~/src/diamond/build/diamond (v2.2.8 + --append + dieser Patch)`, dem Binary, das auch die Cron-Skripte nutzen.

Die BLAST-formatierte nr in `~/blastdb` wird derzeit noch gebraucht: API (`acc`, Abschnitt 7.5) und AmyloDeep (`--acc/--kandidaten`, Abschnitt 8) holen Residuen per `blastdbcmd` daraus. Wie diese doppelte Datenhaltung aufgelöst wird, beschreibt Abschnitt 9.

5. Append: neue Sequenzen ohne Neubau

NCBI bietet für die BLAST-formatierte nr kein inkrementelles Update an (README `blast/db`: „Incremental update is not available“), und DIAMOND kann an eine bestehende `.dmnd` nichts anhängen. Beides wurde ergänzt.

5.1 Patch diamond makedb --append

Branch `makedb-append` in `~/src/diamond`, Diff in `~/src/diamond-makedb-append.patch` (3 Dateien, ~140 Zeilen), Binary `~/src/diamond/build/diamond (v2.2.8 + Patch)`. Aufbau einer `.dmnd`: Header, Sequenzen, danach der Trailer aus Offset-Tabelle, Taxon-Listen (varint-kodiert je OID, ohne eigenen Index), Taxonomie-Knoten und Namen; der Header wird zuletzt per `seek(0)` geschrieben. Ablauf von `--append`:

1. Header lesen und prüfen (Magic, Version 3, gleiche `--taxonmap/--taxonnodes/--taxonnames`-Nutzung wie beim Erstbau).
2. Header und kompletten Trailer in `<db>.append_backup` sichern (bei nr ca. 20–25 GB).
3. Datei am Ende der Sequenzen abschneiden (`pos_array_offset`), neue Sequenzen dahinter schreiben. Der fortlaufende MurmurHash3 läuft vom gespeicherten Wert weiter.
4. Trailer neu schreiben: alte Offset-Einträge + neue, alte Taxon-Listen byteweise + neue (`TaxonList::build` nur für die neuen OIDs), Taxonomie aus dem aktuellen `taxdump`.
5. Header schreiben, Backup löschen. Bei jedem Fehler stellt der Patch Header und Trailer aus dem Backup wieder her.

Tests (23.09.2026): (1) DB aus A (12.000 Sequenzen) plus --append von B (8.000) ist **bytwweise identisch** mit der in einem Durchgang gebauten DB aus A+B (cmp), inkl. Hash, Taxon-Listen und Header. (2) Append mit einer Sequenz der Länge 0 bricht ab, die DB ist danach bytwweise identisch zum Stand vorher, kein Backup-Rest. (3) diamond blastp mit Ausgabe staxids auf der erweiterten DB liefert korrekte Treffer.

5.2 Update-Skript diamond_append.py

Erkennung eines neuen Releases: nr-prot-metadata.json von NCBI (last-updated, number-of-sequences) gegen den neuesten Eintrag in blast_release; gleich = sofort Ende. Bei neuem Release: Volume für Volume laden (md5 geprüft), entpacken, Accessions gegen ein sortiertes 64-bit-Hash-Array aller bekannten Accessions (~15 GB, per Memory-Map) prüfen, Volume löschen. Spitzenbedarf ein Volume statt 855 GB. Neu ist eine Sequenz, wenn keine ihrer Accessions bekannt ist. Danach genau ein --append (Zustandsdatei verhindert doppeltes Anhängen), Metadaten in blast_seq/blast_acc mit neuen OIDs ab dem bisherigen Maximum, neue Zeile in blast_release, Mail an gh@heissa.de. Fortsetzbar je Volume. Wann eine Sequenz hinzukam, liefern die Views blast_seq_herkunft/blast_acc_herkunft (OID-Bereich in blast_import → Release).

Ende-zu-Ende-Test mit Testtabellen blasttest_*: Bestand A, „neues Release“ = Volume nr.000 bis zur 20.000. Sequenz. Ergebnis: genau 8.000 neue Sequenzen erkannt, .dmnd danach bytwweise identisch zur Voll-DB, Metadaten per Prüfsumme identisch zur Produktion, zweiter Lauf erkennt „kein neues Release“.

Grenzen: neue Accessions an bereits vorhandenen Sequenzen (neuer Organismus für dieselbe Sequenz) und von NCBI zurückgezogene Sequenzen werden nur gezählt und erst bei einem Vollneubau berücksichtigt. 64-bit-Hash: rechnerisch ~0,2 fälschlich als bekannt eingestufte Accessions je Release-Abgleich.

Sequenzen, die bereits per Tagesdelta (5.3) angehängt wurden, erkennt das Skript am Sequenz-Hash und hängt sie nicht ein zweites Mal an (Zähler „schon per Tagesdelta“). Der MariaDB-Teil läuft in einer Transaktion (6.4). Test mit Testtabellen: 3.000 Sequenzen, davon 1.000 schon in nr (Accession) und 10 schon per Tagesdelta (Sequenz) → genau 1.990 angehängt, alte Release-Zeilen unverändert.

5.3 Tagesdelta nr_daily_delta.py

Die BLAST-nr wird von NCBI nur als Ganzes neu erzeugt (letzter Stand 16.09.2026, blast/db/FASTA/nr.gz seit 06.02.2024 eingefroren). Zwischen zwei Releases kommen neue Proteine aus den Tagesdateien: genbank/daily-nc/ncMMDD.gnp.gz (GenPept, ~180 Tage vorgehalten) und refseq/daily/rsnc.MMDD.YYYY.gpff.gz (~21 Tage). Das GenPept-Format enthält Accession, Titel, TaxID (/db_xref="taxon:), Sequenz und in der LOCUS-Zeile das NCBI-Datum.

Duplikate nach Sequenz, nicht nach Accession: nr fasst identische Proteine zu einem Eintrag zusammen, führt dabei aber nicht jede Accession auf (Beispiel: YGQ27362.1 ist sequenzgleich zu YP_009724396.1, dessen nr-Eintrag nur 7 Defines hat). Ein Abgleich nach Accession würde also massenhaft Duplikate anhängen. Deshalb: blake2b-64-Hash jeder Sequenz (Großbuchstaben) gegen das sortierte Hash-Array aller nr-Sequenzen (/mnt/archive/nr_seq_hashes.npy, 9,2 GB, per Memory-Map) plus die im selben Lauf schon gesehenen. Alle offenen Tagesdateien werden in **einem** --append verarbeitet, die TaxID-Zuordnung kommt als kleine Mapping-Datei aus dem GenPept selbst.

Test (23.09.2026, Testtabellen, Test-dmnd): nc0919 (19.09.): 5.189 Proteine, 100 schon in der Test-„nr“, 3.195 innerhalb der Datei doppelt, 1.894 neu; nc0920: 7.104 Proteine, 6.491 neu. dmnd danach 100 + 8.385 Sequenzen, OIDs in MariaDB lückenlos, jede neue Sequenz mit NCBI-Datum; zweiter Lauf: „Keine offenen Tagesdateien“. Beispiel einer neuen humanen Sequenz: BIN93201.1 *spexin precursor, partial [Homo sapiens]*, NCBI-Datum 19.09.2026.

Cron: 03:30 diamond_append.py (neues nr-Release), 04:30 nr_daily_delta.py (Tagesdateien). Beide nehmen dieselbe Sperre /home/gh/.nr_dmnd.lock und warten, bis der Erstaufbau (dmnd_hash und meta_importiert in blast_release) abgeschlossen ist. Nach einem neuen nr-Release beginnt das Tagesdelta ab dessen Datum; die zwischenzeitlich per Delta angehängten Sequenzen überspringt der Release-Abgleich per Sequenz-Hash.

6. Metadaten in MariaDB (wagodb, Präfix blast_)

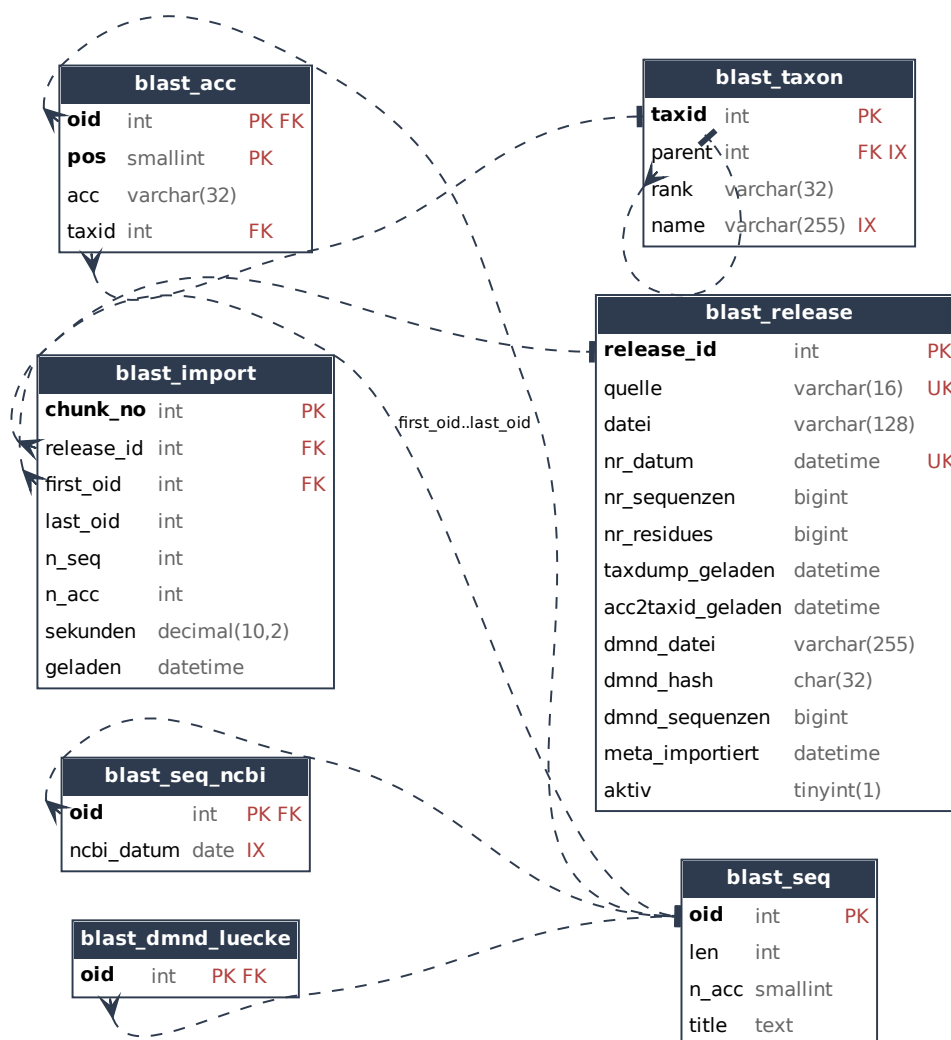
Die .dmnd-Datei kennt nur Sequenzen, Accessions als Namen und TaxIDs – kein Datum, keinen NCBI-Stand, keine Herkunft. Auch die BLAST-nr enthält kein Einreichungsdatum. Fragen wie „welcher nr-Stand ist geladen“, „seit wann ist diese Sequenz in der DB“ oder „wann kam die letzte humane Sequenz hinzu“ lassen sich deshalb nur über die Metadaten in MariaDB beantworten. Sie sind zugleich die Grundlage der API (Treffer aus DIAMOND per Accession mit

Titel, Organismus und Taxonomie anreichern). Jedes Skript, das die .dmnd verändert, pflegt im selben Lauf auch diese Tabellen.

6.1 Tabellen

Tabelle	Zeilen (nr 09/2026)	Inhalt
blast_seq	1,15 Mrd.	je Sequenz: oid (PK), Länge, Zahl der Accessions, Titel der ersten Defline
blast_acc	~1,94 Mrd.	je Defline: oid, pos, acc, taxid; Index auf acc und taxid
blast_taxon	~2,7 Mio.	NCBI-Taxonomie: taxid, parent, rank, wissenschaftlicher Name
blast_release	1 + 1 je Tagesdatei	Stand: quelle (nr / genbank-daily / refseq-daily), datei, nr_datum, Sequenzen, dmnd_hash, aktiv
blast_import	231 + 1 je Delta	OID-Bereich first_oid..last_oid → release_id, Zeitpunkt geladen
blast_seq_ncbi	nur Delta-Sequenzen	oid → NCBI-Datum aus der GenPept-LOCUS-Zeile

blast_seq und blast_acc sind InnoDB ROW_FORMAT=COMPRESSED. Beim Import-Zwischenstand (140 Mio. Sequenzen) belegen beide je 6,3 GB; hochgerechnet auf die ganze nr je ~52 GB, dazu die beiden Indizes auf blast_acc. Alles liegt auf der System-NVMe (/var/lib/mariadb).



ER-Diagramm der nr-Metadaten, erzeugt aus dem Live-Schema (information_schema). Durchgezogen: deklarierte Fremdschlüssel, gestrichelt: logische Verknüpfungen; Krähenfuß = viele. PK/UK/IX/FK wie in der Datenbank; Indizes auf blast_acc erscheinen, sobald nr_build.py sie gebaut hat.

6.2 Herkunft und Datum je Sequenz

Jede Sequenz gehört über ihren OID-Bereich in `blast_import` zu genau einer Zeile in `blast_release`. Die Views `blast_seq_herkunft` und `blast_acc_herkunft` lösen das auf: `quelle`, `datei`, `nr_stand`, `ncbi_datum`, hinzugefügt (Zeitpunkt des Imports). Grenze: alle Sequenzen des Erstimports tragen nur den nr-Stand 16.09.2026, denn die BLAST-nr liefert kein Einreichungsdatum. Ein echtes Tagesdatum haben erst die Sequenzen aus dem Tagesdelta (`ncbi_datum` aus der LOCUS-Zeile).

OID-Nummern: die OIDs in MariaDB sind die BLAST-OIDs, fortgezählt für angehängte Sequenzen. Sie stimmen nicht mit der Satznummer in der `.dmnd` überein (leere Sequenzen beim Bau verworfen). Verknüpft wird deshalb über die Accession (DIAMOND-Ausgabe `sseqid` → `blast_acc.acc`).

6.3 Beispielabfragen

```
-- welcher Stand ist geladen?
SELECT quelle, datei, nr_datum, nr_sequenzen, dmnd_hash
FROM blast_release ORDER BY nr_datum DESC LIMIT 5;

-- wann kam die letzte humane Sequenz hinzu?
SELECT acc, quelle, datei, ncbi_datum, hinzugefuegt
FROM blast_acc_herkunft WHERE taxid = 9606
ORDER BY hinzugefuegt DESC, ncbi_datum DESC LIMIT 1;

-- neue Sequenzen je Tag
SELECT n.ncbi_datum, COUNT(*) FROM blast_seq_ncbi n GROUP BY 1 ORDER BY 1 DESC;

-- DIAMOND-Treffer anreichern
SELECT a.acc, s.title, t.name FROM blast_acc a
JOIN blast_seq s USING (oid) JOIN blast_taxon t ON t.taxid = a.taxid
WHERE a.acc = 'YP_009724390.1';
```

6.4 Wer schreibt was

Skript	Wann	Schreibt
<code>blast_meta2db.py</code>	einmalig (Erstaufbau)	<code>blast_seq</code> , <code>blast_acc</code> in 231 Stücken zu je 5 Mio. Sequenzen (LOAD DATA LOCAL, ~95 s je Stück, fortsetzbar über <code>blast_import</code>), danach Indizes und <code>blast_taxon</code>
<code>blast_release_finalize.sh</code> <code>diamond_append.py</code>	nach Build + Import Cron 03:30, neues nr-Release	<code>dmnd_hash</code> , <code>dmnd_sequenzen</code> , <code>meta_importiert</code> für den Erststand neue <code>blast_release</code> -Zeile (<code>quelle nr</code>), OID-Bereich, <code>blast_seq/blast_acc</code> der neuen Sequenzen
<code>nr_daily_delta.py</code>	Cron 04:30, Tagesdateien	je Tagesdatei eine <code>blast_release</code> -Zeile, OID-Bereich, <code>blast_seq/blast_acc/blast_seq_ncbi</code>

Konsistenz .dmnd ↔ MariaDB: die Append-Skripte hängen erst an die `.dmnd` an und merken sich das in einer Zustandsdatei (`dmnd_done`, Hash, Sequenzzahl). Danach schreiben sie alle Metadaten in **einer** Transaktion. Bricht dieser Teil ab, rollt MariaDB vollständig zurück; der nächste Lauf hängt nicht erneut an, er trägt nur die Metadaten nach. Getestet: bei absichtlich fehlender Tabelle blieb MariaDB auf dem alten Stand, die `.dmnd` wurde nicht doppelt erweitert (Hash unverändert), der Nachlauf ergänzte genau die fehlenden 8.385 Sequenzen. Die Hash-Arrays (Accessions, Sequenzen) werden erst nach dem Commit fortgeschrieben.

Zugangsdaten: alle BLAST-Skripte holen sie über `~/python/blast_db.py` (liest `DB_CFG` aus `~/mqtt-listener.py`, Datenbank `wagodb`).

7. Öffentliche API

Damit auch andere die DIAMOND-Suche und die Metadaten nutzen können, gibt es eine minimale JSON-API: <https://yt.heissa.de/blast/api.php>. `yt.heissa.de` hat nur einen AAAA-Eintrag, die API ist von außen also **nur über IPv6** erreichbar (von `heissa.de` aus getestet). Ohne Parameter liefert sie ihre eigene Beschreibung.

7.1 Endpunkte

Aufruf	Zugang	Liefert
GET ?r=release	frei	geladener nr-Stand, Tagesdeltas seitdem, Bereitschaft von Suche und Metadaten, Warteschlange
GET ?r=acc&acc=YP_009724390.1	frei	Titel, Länge, Herkunft/Datum der Sequenz, TaxID mit Namen, alle identischen Einträge (bis 200). Ohne .Version: neueste Version
GET ?r=taxon&taxid=9606 bzw. &name=Homo sapiens	frei	Taxon mit vollständiger Abstammungslinie
POST ?r=search	X-API-Key	legt einen Suchauftrag an (Query per seq oder acc); Antwort 202 mit job_id, Position und Ergebnis-URL
GET ?r=job&id=...	wer die id kennt	Status, Parameter, Laufzeit, nr-Stand/dmnd_hash der Suche; wenn fertig die Treffer mit Titel und Taxon-Namen aus MariaDB

Suchparameter (Formular oder JSON): seq (FASTA oder rohe Sequenz, bis 50 Sequenzen / 100.000 Buchstaben) oder acc (Accession aus nr, optional mit range von-bis, nur blastp; siehe 7.5), mode blastp oder blastx, taxonlist (bis 20 TaxIDs, z.B. 4751 = Fungi, 9606 = Mensch), evalute (Standard 0,001), max_target_seqs (1–500, Standard 25), sensitivity fast bis ultra-sensitive (Standard sensitive).

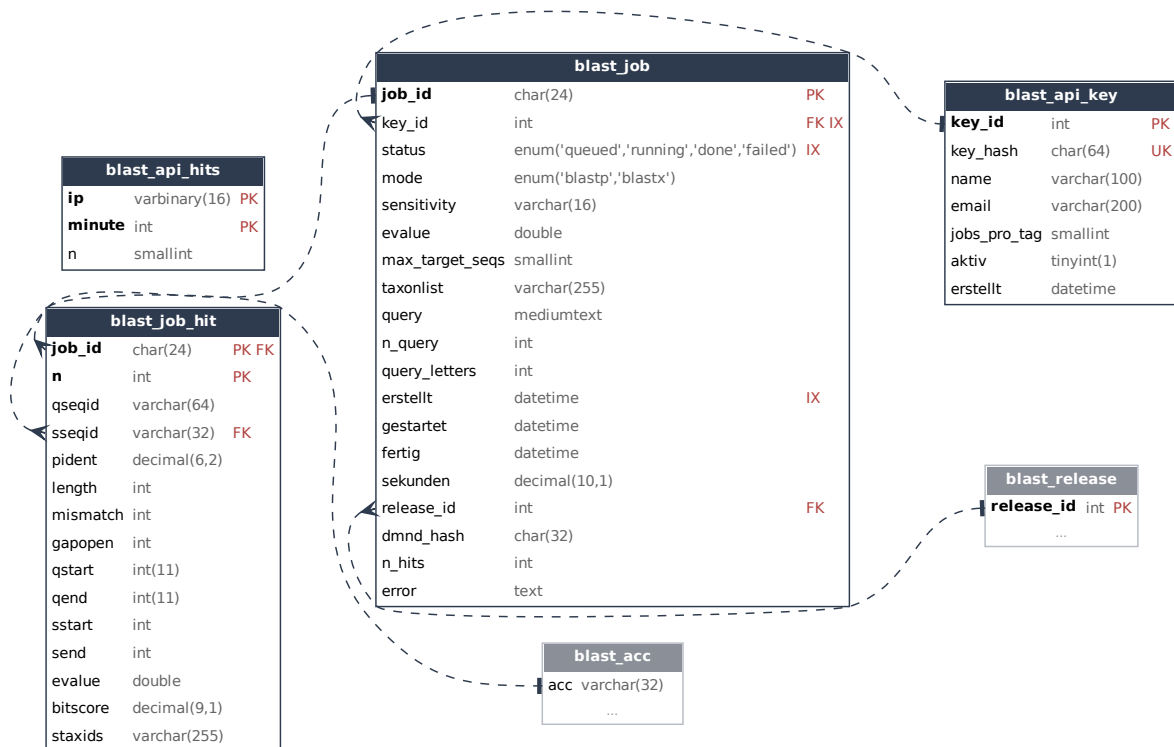
```
curl -X POST -H 'X-API-Key: ...' -H 'Content-Type: application/json' \
-d '{"seq": ">q1\nMKT...", "mode": "blastp", "taxonlist": "9606"}' \
'https://yt.heissa.de/blast/api.php?r=search'

curl 'https://yt.heissa.de/blast/api.php?r=job&id=18a8ceae9fcc7d63063e5ffc'
```

7.2 Aufbau

api.php (/var/www/blast, mod_php, Alias im vHost yt.heissa.de vor dem Invidious-Proxy) beantwortet Metadaten-Anfragen direkt und legt Suchen nur als Zeile in blast_job an. Bei acc holt sie die Query-Sequenz vorher per blastdbcmd -entry ... -target_only [-range ...] aus der BLAST-nr (~blastdb/nr, ~0,3 s). Eine DIAMOND-Suche gegen die volle nr belegt alle 20 Kerne für Minuten und darf deshalb nie im Webserver-Prozess laufen. **blast_api_worker.py** (systemd blast-api-worker, Benutzer gh) holt die Aufträge nacheinander, ruft DIAMOND auf (Zeitlimit 1 h), schreibt die Treffer in blast_job_hit und vermerkt nr-Stand und dmnd_hash. Während der Suche hält er eine geteilte Sperre auf /home/gh/.nr_dmnd.lock; die Append-Skripte brauchen die exklusive Sperre und verändern die .dmnd daher nie unter einer laufenden Suche. Nach einem Absturz setzt er hängende Aufträge wieder auf queued. Solange dmnd_hash im aktiven Release fehlt (Erstaufbau), nimmt die API keine Suchen an (503).

Tabelle	Inhalt
blast_api_key	Schlüssel nur als sha256, Name, E-Mail, Tageskontingent, aktiv
blast_job	Auftrag: Parameter, Query, Status, Zeiten, Laufzeit, release_id, dmnd_hash, Trefferzahl, Fehler
blast_job_hit	Treffer im DIAMOND-Format 6 (qseqid ... bitscore, staxids)
blast_api_hits	Zähler je IP bzw. IPv6-/64 und Minute (Ratenbegrenzung), vom Worker aufgeräumt



ER-Diagramm der API (grau: Tabellen aus Abschnitt 6, nur verknüpfte Spalten), erzeugt aus dem Live-Schema (information_schema). Durchgezogen: deklarierte Fremdschlüssel, gestrichelt: logische Verknüpfungen; Krähenfuß = viele. PK/UK/IX/FK wie in der Datenbank.

7.3 Schutz

Eigener MariaDB-Benutzer `blast_api`: nur SELECT auf die Metadaten-Tabellen und Views, INSERT nur auf `blast_job`, INSERT/UPDATE nur auf `blast_api_hits`; Zugangsdaten in `/etc/blast_api.ini` (root:www-data, 640). Alle Abfragen mit Prepared Statements, Eingaben per Muster geprüft; `blastdbcmd` wird ohne Shell mit geprüfter Accession/Range aufgerufen. 60 Anfragen je Minute und Client. Suchen nur mit Schlüssel, höchstens 3 offene Aufträge je Schlüssel, Tageskontingent je Schlüssel (Standard 20). Die `job_id` ist 96 bit Zufall und zugleich die Berechtigung zum Lesen des Ergebnisses. Accession-Abfragen antworten mit 503, bis der Index `idx_acc` existiert – ohne ihn wäre jede Abfrage ein Scan über 1,9 Mrd. Zeilen.

Schlüssel verwalten (auf dem dell): `python3 ~/python/blast_api_key.py add NAME [EMAIL] [JOBS_PRO_TAG]` gibt den Schlüssel einmalig aus; `list` zeigt alle mit Zahl der Aufträge; `off KEY_ID` sperrt einen.

7.4 Test (23.09.2026)

Von außen (heissa.de, IPv6): Beschreibung, release, Fehlerfälle (kein/falscher Schlüssel, ungültige Sequenz, ungültige id, Index fehlt) liefern die erwarteten Codes. Suchweg Ende zu Ende gegen eine Test-dmnd (Tagesdatei nc0920, 6.491 Sequenzen): `blastp` der humanen Sequenz BIN93201.1 mit `taxonlist=9606` → 1 Treffer, 100 % Identität, E-Wert $3e-72$, 0,5 s. Titel und Taxon-Namen erscheinen, sobald der Metadatenimport Index und Taxonomie geladen hat. Testdaten danach entfernt. Der `acc`-Weg ist noch nicht Ende zu Ende gelaufen: solange `diamond makedb` läuft, antwortet die Suche mit 503.

7.5 Beispiel: SARS-CoV-2-Spike-RBD gegen humane Proteine

Frage: hat die Rezeptorbindedomäne (RBD) des Spike-Proteins Ähnlichkeit mit menschlichen Proteinen? Übergeben wird nur die **Query**, also die Sequenz, zu der Treffer gesucht werden. Die Vergleichsseite ist immer die komplette nr (`nr_full.dmnd`); `taxonlist=9606` beschränkt die Treffer auf *Homo sapiens*, `mode=blastp` vergleicht Protein gegen Protein.

Variante A – Query per Accession. Die Spike-Sequenz liegt selbst in nr (YP_009724390.1, 1.273 aa). Die RBD sind die Reste 319–541 (223 aa); die API schneidet sie per range heraus, die Query heißt dann YP_009724390.1_319–541:

```
curl -X POST -H 'X-API-Key: YOUR_API_KEY' -H 'Content-Type: application/json' \
-d '{"acc": "YP_009724390.1", "range": "319-541", "mode": "blastp",
  "taxonlist": "9606", "max_target_seqs": 25}' \
'https://yt.heissa.de/blast/api.php?r=search'
```

Variante B – Query als Sequenz, z.B. für eine eigene oder noch nicht in nr enthaltene Variante. Dieselbe RBD als FASTA; Zeilenumbrüche in der Sequenz sind erlaubt:

```
cat > rbd.fa <<'EOF'
>SARS-CoV-2_Spike_RBD
RVQPTESIVRFPNITNLCPFGVEFVNATRFASVYAWNKRKISNCVADYSVLVNSASFSTFK
CYGVSPTKLNDLCFTNVYADSFVIRGDEVQRQIAPGQTGKIADYNYKLPDDFTGCVIAWNS
NNLDSKVGGNNYLYRLFRKSNLKPFRDISTEIYQAGSTPCNGVEGFNCYFPLQSYGFQ
PTNGVGYQPYRVVVLSEFLLHAPATVCGPKKSTNLVKNKCVNF
EOF
curl -X POST -H 'X-API-Key: YOUR_API_KEY' \
--data-urlencode seq@rbd.fa -d mode=blastp -d taxonlist=9606 -d max_target_seqs=25 \
'https://yt.heissa.de/blast/api.php?r=search'
```

Als JSON geht es ebenso, dann steht die Sequenz in einer Zeile im Feld seq (FASTA-Kopf mit \n davor). seq und acc zusammen werden abgelehnt (400); eine Accession, die nicht in der BLAST-nr vom 16.09.2026 steht, ergibt 404 – dann Variante B nehmen.

Antwort und Ergebnis: der POST liefert sofort 202 mit job_id, Position in der Warteschlange und Ergebnis-URL; diese abfragen, bis status = done ist:

```
{ "job_id": "...", "status": "queued", "position": 1,
  "url": "https://yt.heissa.de/blast/api.php?r=job&id=..." }

curl 'https://yt.heissa.de/blast/api.php?r=job&id=...'
```

Jeder Treffer enthält die DIAMOND-Spalten (sseqid, pident, length, qstart–qend, sstart–send, evalue, bitscore) plus Titel und Taxon-Namen aus MariaDB. Aussagekräftig sind vor allem E-Wert und Identität über die ausgerichtete Länge; kurze Treffer mit E-Wert nahe 0,001 sind meist Zufall.

8. AmyloDeep auf der Tesla P4

Zweite Anwendung auf denselben Sequenzen: Vorhersage der Amyloid-Neigung mit Protein-Sprachmodellen. AmyloDeep 0.3.1 (Davtyan, Khachatryan & Petrosyan, bioRxiv 2025, doi 10.1101/2025.09.16.676495; PyPI amylodeep, Gewichte AlisaDavtyan/amylodeep-models auf Hugging Face) ist ein Ensemble aus fünf Modellen: ESM-2 150M (feinjustiert), UniRep mit Klassifikationskopf, ESM-2 650M (Mean-Embedding) mit Kopf sowie SVM und XGBoost auf denselben 650M-Embeddings. Die Einzelwahrscheinlichkeiten werden kalibriert (Platt bzw. isotonisch) und gemittelt. Bewertet wird ein Rollfenster (Standard 10 Reste, Schritt 1); jedes Fenster wird auf 128 Token aufgefüllt.

8.1 Passt es in die P4?

Modell	Gewichte FP32	auf der P4
ESM-2 150M (feinjustiert)	~0.6 GB	ja
ESM-2 650M (esm2_t33_650M_UR50D)	~2.6 GB	ja
zusammen, dazu llama-server (0,8 GB)	~4,1 GB von 7,5 GB	ja, FP32, kein FP16 nötig
zum Vergleich: ESM-2 3B	~11,4 GB (FP16 ~5,7 GB)	nein bzw. nur ohne llama-server

UniRep läuft über JAX auf der CPU, SVM und XGBoost ebenfalls; die Torch-Modelle liegen auf der GPU.

8.2 Umgebung: Env iver mit CUDA 11.8

Das micromamba-Env iver (~/.iver_sim/mamba/envs/iver) war auf CUDA 11.8 gepinnt, enthielt aber nur einen **CPU-Build** von PyTorch 2.5.1. Ersetzt durch den conda-forge-Build pytorch 2.5.1 cuda118; dessen Architekturliste

enthält `sm_61` nativ, ein eigener Compile war nicht nötig. Dazu kamen `cudnn`, `magma` und `nccl`; `libprotobuf` 5.28.3 → 5.28.2.

Der nächste Install hätte PyTorch sofort wieder auf 2.10 CPU und `numpy` auf 2.x gezogen. Deshalb stehen in `conda-meta/pinned` jetzt zusätzlich `pytorch 2.5.1 cuda118*`, `libtorch 2.5.1 cuda118*` und `numpy 1.26.*` (Sicherheit `pinned.bak-20260923`). Installiert: `transformers 5.16.1`, `huggingface_hub 1.32`, `jax/jaxlib 0.5.2` (CPU), `xgboost 3.4.2` (CPU), `equinox`, `optax`, `optuna`; `amylodeep 0.3.1` und `jax-unirep 3.0.0` per `pip` mit `--no-deps`. `openmm.testInstallation` danach unverändert fehlerfrei (CUDA und OpenCL).

Ausführbarer Stack: Kernel 7.0 mit `glibc 2.43` lädt keine Bibliotheken mehr, die einen ausführbaren Stack anfordern (`cannot enable executable stack`). Betroffen war nur `jaxlib/xla_extension.so`: im Programmkopf `PT_GNU_STACK` das X-Bit gelöscht (RWE → RW), Original unter `~/iver_sim/xla_extension.so.orig`. Nach einem `jaxlib-Update` ist das zu wiederholen.

8.3 amylo_gpu.py

`~/iver_sim/ol_amyloid/amylo_gpu.py` erbt von `EnsembleRollingWindowPredictor`, ohne das Paket zu verändern. Das Paket selbst rechnet nur auf der CPU und führt ESM-2 650M für dieselben Fenster dreimal aus (NN-Kopf, SVM, XGBoost). Der Wrapper legt die Torch-Modelle auf CUDA, rechnet in Stapeln zu 64 und die 650M-Embeddings je Fenstermenge nur einmal. Eingabe: Sequenz, FASTA, `--acc` oder `--kandidaten` (TSV Accession, Kürzel); Sequenzen per `blastdbcmd` aus der lokalen `nr` (siehe Abschnitt 9).

Prüfung gegen das Original (`--vergleich`), A₃₄₂ (42 aa, 33 Fenster): größte Abweichung je Fenster 7e-7, GPU 4,4 s gegen 43,4 s CPU. Die stärksten Fenster liegen auf den bekannten Aggregationskernen: GLMVGGVVIA 0,91 (C-Terminus), KLVFFAEDVG 0,90 (KLVFF, Reste 16–20); am schwächsten der N-Terminus DAEFRHDSGY 0,07.

```
cd ~/iver_sim/ol_amyloid
PY=~/iver_sim/mamba/envs/iver/bin/python
$PY amylo_gpu.py --import-tsv kontrolle.tsv --rolle kontrolle      # Kandidaten anlegen (acc, kurz, name)
$PY amylo_gpu.py --kandidaten amyloid --bemerkung "43 amyloid candidates"
$PY amylo_gpu.py --acc YP_009724390.1 -w 10
```

8.4 Tabellen in wagodb (Präfix amy1_)

Schema `~/iver_sim/ol_amyloid/amy1_schema.sql`. Jede Angabe steht genau einmal: Kandidat → Sequenzversion → Ergebnis je Lauf → Fenster. Fenstertexte, Fensterzahl und NCBI-Titel/Taxon werden nicht gespeichert, sondern in den Views abgeleitet. Die Kandidatenliste lebt in `amy1_kandidat`, nicht mehr in einer TSV-Datei.

Tabelle	Inhalt
<code>amy1_kandidat</code>	je Protein einmal: Kürzel, Name, Accession ohne Version, Rolle (amyloid / kontrolle / einzel), reife Kette von–bis (NULL = ganze Sequenz), Bemerkung
<code>amy1_sequenz</code>	je Sequenzversion einmal: Kandidat, versionierte Accession (wie <code>blast_acc</code>), Quelle, Länge, Sequenz, SHA1
<code>amy1_lauf</code>	je Aufruf: Start, Laufzeit, Host, GPU, Software-Stände, Fenstergröße, Zahl der Sequenzen, Bemerkung
<code>amy1_ergebnis</code>	je Lauf und Sequenz: bewerteter Bereich, Mittel, Maximum, Position des Maximums, Laufzeit
<code>amy1_fenster</code>	je Fenster: Position (1-basiert, auf die ganze Sequenz bezogen), Wahrscheinlichkeit
<code>amy1_v_ergebnis</code> , <code>amy1_v_fenster (Views)</code>	Ergebnisse mit Kandidat, stärkstem Fenster (SUBSTRING), Fensterzahl sowie Titel und Taxon aus <code>blast_acc/blast_seq/blast_taxon</code>

Kürzel	Accession	Länge	Mittel	Max	Pos.	stärkstes Fenster
Alns	P01308.1	110	0,45	0,96	94	QCCTSICSLY
AApoCIII	P02656.1	99	0,36	0,96	5	VLLVVALLAL
AGel	P06396.1	782	0,34	0,96	184	VVVQRLFQVK
ALys	P61626.1	148	0,42	0,96	8	GLVLLSVTVQ
AApoAIV	P06727.4	396	0,28	0,96	5	AVVLTALVA
AIAPP	P10997.1	89	0,51	0,95	9	FLIVLSVALN
AApoAII	P02652.1	100	0,42	0,95	84	VNFLSYFVEL
AMed	Q08431.3	387	0,43	0,95	313	VQFVASYKVA
AB2M	P61769.1	119	0,41	0,95	5	VALAVLALLS
APro	P01236.1	227	0,42	0,95	108	FLSLIVSILR
ABri	Q9Y287.1	266	0,43	0,95	62	AFMLAGVILG
ALac	P02788.6	710	0,36	0,95	5	FLVLLFLGAL
APrP	P04156.1	253	0,35	0,95	240	VILLISFLIF
AApoCII	P02655.1	101	0,43	0,95	9	LFLVLLVLGF
AFib	P02671.2	866	0,28	0,95	6	IVCLVLSVVG
ACal	P01258.3	141	0,35	0,95	10	LALSILVLLQ
AANF	P01160.2	151	0,26	0,95	10	SFLLLLAFQL
CsgA	P28307.3	151	0,43	0,94	5	KVAAIAAIVF
ATTR	P02766.1	147	0,42	0,94	8	LLCLAGLVFV
ASem1	P04279.2	462	0,28	0,94	432	GGLDIVIEEQ
SOD1	P00441.2	154	0,35	0,94	145	LACGVIGIAQ
AL	P01834.2	107	0,36	0,94	24	SVVCLLNIFY
AA	P0DJ18.2	122	0,31	0,94	4	LTGLVFCSLV
ALECT2	O14960.2	151	0,38	0,93	97	GFCVKMFYIK
AaSyn	P37840.1	140	0,36	0,93	70	VVTGVTAVAQ
SEVI	P15309.3	386	0,38	0,92	258	LQGGVLVNEI
ATau	P10636.5	758	0,16	0,92	619	GGGSVQIVYK
HETS	Q03689.3	289	0,37	0,92	5	FGIVAGALNV
ACor	Q15517.4	529	0,31	0,90	16	GMMALLLAGL
AGal7	P47929.2	136	0,35	0,89	32	RFHVNLLCGE
FUS	P35637.1	526	0,31	0,88	140	QQSYGQQQSY

8.6 API und Worker (amylo-api-worker)

AmyloDeep hängt nicht mehr an einer SSH-Sitzung: dieselbe JSON-API wie für DIAMOND nimmt Aufträge entgegen, mit denselben Schlüsseln, derselben Ratenbegrenzung und demselben Warteschlangen-Muster. Der Grund für einen eigenen Worker ist das Modell: es einmal zu laden kostet 6,5 s, und ein Aufruf von `amylo_gpu.py` zahlt das jedes Mal. Der Worker (systemd `amylo-api-worker`, Env `iver`) hält ESM-2 150M/650M, UniRep, SVM und XGBoost dauerhaft auf der Tesla P4 (4,6 GB) und arbeitet die Aufträge nacheinander ab.

Aufruf	Zugang	Liefert
POST ?r=amylo	X-API-Key	legt einen Auftrag an: <code>acc</code> (aus nr, optional von/bis) oder <code>seq, window 4–40</code> (Standard 10); Antwort 202 mit <code>job_id</code> und Ergebnis-URL
GET ?r=amylojob&id=...	wer die id kennt	Status, Parameter, Laufzeit; wenn fertig Mittel, Maximum, stärkstes Fenster und alle Fensterwerte

Grenzen wie bei der Suche: höchstens 3 offene Aufträge und ein Tageskontingent je Schlüssel, 60 Anfragen je Minute, bis 5.000 Reste je Auftrag. Der Benutzer `blast_api` darf `amy1_job` schreiben und die Ergebnistabellen nur lesen. Jeder Auftrag erzeugt einen eigenen `amy1_lauf`, die Werte landen in denselben Tabellen wie die Läufe von Hand (8.4) – ein API-Ergebnis ist damit später genauso auswertbar.

8.7 Beispiel: eine Literaturaussage prüfen (Spike 194–203)

Der Fall zeigt, was die Teile zusammen leisten. Ausgangspunkt ist eine Angabe aus der Literatur (Nyström & Hammarström, *JACS* 2022, hier als Vorgabe übernommen und **nicht** nachgeprüft): im SARS-CoV-2-Spike gebe es sieben amyloidogene Abschnitte, und das Fragment **194–203** bilde nach Spaltung durch neutrophile Elastase Fibrillen. Geprüft wird nur, ob AmyloDeep diesen Abschnitt auffällig findet; Spaltstellen sagt das Modell nicht vorher.

Aus 8.5 folgt die Methodik: ein hoher Wert allein sagt nichts, weil alle 43 bekannten Amyloidbildner zwischen 0,88 und 0,97 lagen. Es braucht zwei Bezugsgrößen: **intern** den Rang des Fensters 194 unter allen Fenstern des Spike, **extern** die

Verteilung der Fenster eines Kontrollsatzes zufälliger humaner SwissProt-Proteine (100–800 aa, bekannte Amyloidbildner ausgeschlossen, feste Zufallszahl).

~/iver_sim/ol_amyloid/spike_amyloid_test.py legt Kandidaten und Kontrollen an und wertet danach aus; gerechnet wird über die API (8.5), der Worker hält das Modell bereit:

```
# Kandidat und 300 SwissProt-Kontrollen anlegen
python3 spike_amyloid_test.py --vorbereiten --n-kontrollen 300 --seed 42

# Auftrag ueber die API (Abschnitt 8.5) - kein SSH, kein Modell-Laden
curl -X POST -H "X-API-Key: $K" -H 'Content-Type: application/json' \
-d '{"acc": "YP_009724390.1", "window": 10}' \
'https://yt.heissa.de/blast/api.php?r=amylo'
-> {"job_id": "0f478cae...", "status": "queued", "position": 1, "url": "...r=amylojob&id=..."}

curl 'https://yt.heissa.de/blast/api.php?r=amylojob&id=0f478cae...'
python3 spike_amyloid_test.py --auswerten
```

Was dabei ineinandergreift – und warum das ohne den Unterbau der vorigen Abschnitte Tage statt Minuten dauern würde:

Schritt	Grundlage	Aufwand
Spike-Sequenz und 300 Kontrollsequenzen holen	lokale nr per Accession (Abschnitt 4, 9)	~0,3 s je Sequenz, kein Download
Kontrollen auswählen	swissprot_titel.tsv lokal, Filter nach Organismus, Länge und Titel	15.603 Kandidaten, Auswahl per Zufallszahl reproduzierbar
Amyloid-Neigung je Fenster	ESM-2 650M + Ensemble auf der Tesla P4 (8.1–8.3), eingereicht über die API (8.5)	1.264 Fenster in 136 s; GPU 100 %, 78 °C
Ergebnisse ablegen	amyl_* in wagodb, je Angabe einmal (8.4)	Auswertung als SQL statt als Dateiwust
Nachvollziehbarkeit	amyl_lauf hält Zeit, GPU, Software-Stände, Fenstergröße fest	jeder Wert im PDF ist auf einen Lauf zurückführbar

Ergebnis (Lauf 2, 1273 aa, Fenster 10): das Fenster ab Position 194 (FKNIDGYFKI) erreicht 0,630 und liegt damit auf Rang 460 von 1.264 Fenstern des Spike-Proteins, also über 64 % der übrigen Abschnitte.

Rang	Position	Fenster	Wert
1	1218	LGFIAGLIAI	0,971
2	606	NQVAVLYQDV	0,964
3	607	QVAVLYQDVN	0,958
4	1059	GVVFLHVTYV	0,958
5	1222	AGLIAIVMVT	0,953

Die Vorhersage stützt die Angabe nicht. Das Fragment liegt im Mittelfeld des eigenen Proteins, nicht an der Spitze. Die höchsten Werte stehen auf hydrophoben Abschnitten der Transmembran- und Fusionsregion (1218, 1222, 1059) – dasselbe Muster wie bei den 43 Kandidaten, wo 19 Maxima im Signalpeptid lagen. Mögliche Gründe, keiner davon hier geprüft: AmyloDeep bewertet ein Fenster für sich, nicht das Verhalten eines durch Elastase freigesetzten Peptids; die zitierte Arbeit ist experimentell, nicht vorhersagend; und andere Verfahren (WALTZ, AGGRESCAN, ZipperDB) können anders urteilen. Belegt ist nur: **dieses** Modell findet **diesen** Abschnitt unauffällig.

Der SwissProt-Kontrollsatz (300 Proteine) ist angelegt, aber noch nicht gerechnet; er würde zeigen, wie ein Wert von 0,63 im Vergleich zu beliebigen humanen Abschnitten einzuordnen ist.

9. Doppelte Datenhaltung auflösen

Dieselben Sequenzen liegen zweimal auf dem dell: als BLAST-nr und als DIAMOND-Datenbank.

Datenbestand	Größe	Ort	Stand
~/blastdb/nr (BLAST, 176 Volumes)	857 GB	System-NVMe (78 % belegt)	eingefroren 16.09.2026
nr_full.dmnd (DIAMOND)	Neubau läuft (ohne Titel, erwartet ~460 GB)	/mnt/archive (sda5)	per Cron fortgeschrieben (5.2, 5.3)
Metadaten blast_*	2 × ~52 GB + Indizes (hochgerechnet, 6.1)	MariaDB (NVMe)	per Cron fortgeschrieben (6.4)

Suche, Append und Tagesdelta brauchen die BLAST-nr nicht mehr (die Hash-Arrays für den Duplikatabgleich sind gebaut). Übrig ist ein einziger Zweck: **Residuen zu einer Accession holen** (API `acc`, `amylo_gpu.py`). Die Metadaten in MariaDB helfen dabei nicht, `blast_seq` enthält Länge und Titel, aber keine Sequenz. Dazu kommt ein Fehler im Ist-Zustand: die Cron-Skripte hängen neue Sequenzen nur an `.dmnd` und MariaDB an, nicht an die BLAST-nr. Eine per Tagesdelta hinzugekommene Accession steht in `blast_acc`, `blastdbcmd` findet sie aber nicht.

9.1 Titel liegen dreifach vor

Die BLAST-nr besteht nur zu gut der Hälfte aus Sequenzen: `.psq` 436 GB (1 Byte je Rest), `.phr` 270 GB Kopfzeilen, ~155 GB Accession- und Taxon-Indizes (`nr.pdb` als LMDB allein 99 GB), ~45 GB OID-Indizes. Grund: `nr` fasst identische Sequenzen zusammen, behält aber jede Defline – 1,15 Mrd. Sequenzen, 1,94 Mrd. Deflines; ein Titel ist im Mittel ~139 Byte, eine Sequenz 377 Reste.

Die `.dmnd` enthält dieselben Titel noch einmal. `diamond_pipeline.sh` ruft `blastdbcmd -entry all` ohne `-outfmt` auf; das liefert FASTA mit der zusammengefassten Kopfzeile aller identischen Einträge, getrennt durch „>“ (Spike YP_009724390.1: 29 Deflines, 2.634 Byte Titel zu 1.273 Resten). `FastaFile` übernimmt die ganze Zeile als ID (`src/data/fasta/fasta_file.cpp:231`), `push_seq` schreibt sie vollständig hinter die Residuen (0xFF, Residuen, 0xFF, ID, Nullbyte; `src/legacy/dmnd/dmnd.cpp`). Gebraucht werden davon nur die Accessions: `accession_from_title` zerlegt die Zeile an „\1 bzw. „>“ (FASTA_HEADER_SEP, `src/util/sequence/sequence.cpp:38`) und füllt damit die Taxon-Listen. Grob 120 GB der `.dmnd` sind Titeltitel (genaue Zahl nach Abschluss des Builds aus dem Trailer). Die dritte Kopie ist `blast_seq.title` in MariaDB (nur der erste Titel je Sequenz).

Umgesetzt im Neubau vom 23.09.2026 (`nr_build.py`, **Abschnitt 4**): Kopfzeile nur noch aus den Accessions (`>acc1 >acc2 ...`). Taxon-Listen bleiben vollständig, die Suchausgabe `sseqid` ist ohnehin nur das erste Wort, Titel kommen für die API aus MariaDB. Die `.dmnd` wird dadurch rund 100 GB kleiner; die Titel liegen danach nur noch zweimal vor (`.phr` und `blast_seq.title`), nach dem Löschen der BLAST-nr einmal.

9.2 Warum nicht diamond getseq

`diamond getseq (SequenceFile::get_seq(), src/data/sequence_file.cpp:395)` hat keinen Index: die Schleife liest alle `sequence_count()` Sätze der Reihe nach und gibt die ausgewählten aus. `--seq` erwartet Satznummern, nicht Accessions; Titel-Listen werden ebenfalls nur während dieses Durchlaufs verglichen. Für eine einzelne Accession hieße das 495 GB lesen – Stunden statt Millisekunden.

9.3 Direktzugriff: dmnd_getseq.py

Die `.dmnd` hat alles, was für einen Direktzugriff nötig ist: im Trailer liegt ab `pos_array_offset` (Header) die Offset-Tabelle, 16 Byte je Satz (SeqInfo: `pos u64, seq_len u32`, `src/data/sequence_file.h:130`). Geplantes Skript, reines Python, ohne DIAMOND-Aufruf:

1. Accession → OID über `blast_acc` (Index `idx_acc`). Deckt Erstimport, Release-Append und Tagesdelta ab.
2. OID → Satznummer der `.dmnd`: OID minus Zahl der Leersequenzen mit kleinerer OID. Die Leersequenzen schreibt `nr_build.py` beim Bau in `blast_dmnd_luecke`; angehängte Sequenzen erzeugen keine neuen Lücken, `--append` lehnt Länge 0 ab (5.1).
3. Satznummer → Offset: ein seek auf `pos_array_offset + 16·n`, 16 Byte lesen.
4. Offset → Residuen: ein seek, `seq_len` Bytes lesen, DIAMOND-Buchstabencode in Aminosäuren zurückübersetzen.
5. Kontrolle bei jedem Zugriff: hinter den Residuen steht die Kopfzeile des Satzes (9.1); die gesuchte Accession muss darin vorkommen, und `seq_len` muss `blast_seq.len` entsprechen. Sonst stimmt die Lücken-Tabelle nicht – Fehler statt falscher Sequenz.

Kosten: zwei Zufallszugriffe auf `sda5`, Millisekunden. Die Sequenzen sind damit immer auf dem Stand der fortgeschriebenen `.dmnd`, auch für Accessions aus dem Tagesdelta. Während eines Appends ändert sich die Offset-Tabelle; das Skript nimmt deshalb dieselbe geteilte Sperre `/home/gh/.nr_dmnd.lock` wie der API-Worker.

9.4 Umstellung und Freigabe

1. `dmnd_getseq.py` bauen (nach Abschluss von `nr_build.py`; `blast_dmnd_luecke` und `idx_acc` liegen dann vor).
2. Abgleich: für eine Stichprobe von 100.000 zufälligen Accessions `blastdbcmd` gegen `dmnd_getseq.py`, alle Residuen identisch; dazu alle 43 Amyloid-Kandidaten.
3. `api.php (acc)` und `amylo_gpu.py (--acc, --kandidaten)` auf `dmnd_getseq.py` umstellen.
4. Danach `~/blastdb` löschen: 857 GB frei auf der System-NVMe (heute 78 % belegt). Löschen erst nach bestandem Abgleich und ausdrücklicher Freigabe; ein Vollneubau lädt die `nr` ohnehin neu von NCBI.
5. `nr_full.dmnd` danach von der HDD (`sda5`) auf die freie System-NVMe verschieben: DIAMOND streamt die ganze Datenbank je Suche, von der HDD dauert ein Durchgang über ~460 GB eine Stunde und mehr.

10. Mögliche nächste Stufe

Falls DIAMOND bei sehr entfernten Homologien nicht ausreicht: Zwei-Stufen-Suche über Protein-Embeddings, die dasselbe Muster wiederverwendet, das bereits für die Wissensbasis (`vec_`-Tabellen in `wagodb`, MariaDB VECTOR mit HNSW-Index, Embedding-Server auf der P4) produktiv läuft: Sequenzen als Vektoren ablegen, per `VEC_DISTANCE_COSINE` vorfiltern, nur die Kandidaten anschließend exakt mit DIAMOND/BLAST nachprüfen. Die ESM-2-650M-Umgebung aus Abschnitt 8 liefert solche Embeddings bereits.